Resource Estimation for Web Applications

Paul Umbers
Intemistic Ltd, Birmingham, UK
paul.umbers@bcs.org.uk

Dr. Gail Miles
Lenoir-Rhyne College, North Carolina, USA
gait.miles@embanet.com

Abstract

In the field of Software Engineering, several empirical methods have been developed to model software projects before they are undertaken to provide an estimate of required effort, development time and cost. In the case of web applications, this process is complicated by their complexity, multi-tiered nature, extensive use of non-code artifacts such as multimedia and often short time-scales. In this paper we describe a simple, highly adaptable model using COSMIC Full Function Points for application size measurement and design patterns as a measurement reference. Rather than a true derived model the aim is to provide a procedural framework for expert judgement which guides the practitioner through the estimation process, seeking to limit or mitigate variance in their judgement through algorithmic or statistical techniques. This hybrid has so far proven as accurate as expert judgement while remaining capable of application by a relatively inexperienced estimator.

1. Introduction

Web Applications (a term in which we include Web Services[l]) are generally considered among the most difficult software constructs to size and estimate. The prevalence of architectural layers (often integrating disparate legacy systems), their atypical structure and implementation from one project to another and the preponderance of non-code artifacts (such as multimedia objects) make the definition of descriptive metrics extremely difficult.

Mendes et **al** discuss several different measures[2, 3], however in the early stages of a design many of the measured artifacts are either unknown or poorly defined and a design detailed enough to measure may well be considered too time-consuming to produce, particularly in the case of small projects. Many teams adopt a more ad-hoc and intuitive approach, but this has the major drawback that the result tends to be unreliable and difficult to repeat accurately.

The aim of this project was to develop a methodology which could be used by small development teams with little or no previous experience of estimation but which was accurate enough for repeated commercial application where more detailed methods would be considered overkill.

In particular, we are interested in predicting the effort required to complete work defined by the Rational Unified Process (RUP) phases when applied to small projects or small teams. Such projects are often characterised by the level of formality applied to the processes, the complexity of the code and the timescale for completion[4].

In developing the project, we defined four basic principles: the use of COSMIC Full Function Points[5] as a sizing metric, the use of design patterns as a way of simplifying the measurement process for highly iterative projects, the use of intuitive indicators of application complexity which can be easily related to real-world development and the use of Monte Carlo Simulation to help mitigate errors of judgement.

2. Sizing using Design Patterns and COSMIC-FFP

If we consider application structure alone for the moment, COSMIC Full Function Points[5], with it's concepts of layers, boundaries, functional processes and data movements appears the most suited to sizing web applications as similar concepts are present in the multi-tiered architecture seen in most projects[6].

However, trying to break an application down into functional processes sufficient for COSMIC measurement can be a laborious task, particularly if it's design has not been finalised.

Current development practice is that many enterprise applications use standard *designpatterns[7]* which exhibit relatively consistent functional characteristics

Proceedings of the 10th International Symposium on Software Metrics (METRICS'04)
1530-1435/04 $ 20.00 IEEE
Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.
IEEE COMPUTER SOCIETY

such that the measurement process can be simplified and abstracted somewhat while still producing an acceptably accurate and repeatable result. In most cases, the number of data movements is constant for a given pattern, while it is the number of data groups moved which varies. They are therefore useful as a shortcut which **will** allow us to produce a reasonable estimate of application size from a relatively simple interaction model early **in** the development process.

In the case of web applications, the *Model- View-Controller, Intercepting Filter* and *Data Access Object* patterns are commonly combined in a variety of application frameworks (such as Jakarta Struts[S] for the J2EE architecture) and provide a good starting point for our initial study. Each of these design patterns implements one or more functional processes (within the COSMIC FFP definition) which we will specify and/or identify for measurement purposes. Figure 1 shows how these components typically interact in a web applica**tion.**

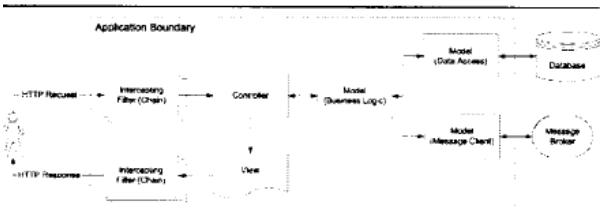

Figure 1. How filter, model, view and controller components interact in web applications.

2.1. Application Design

Given a set of functional user requirements, we can identify interactions between end-users (including other systems) across the application boundary. These can be represented in a variety of ways according to personal preference or local practice. The Web Application Extension for UML[9, 10], for example, can be used to chart interactions from the user perspective (UX Model) which can later be expanded into Design and Implementation models showing architectural features.

From the UX model we can identify where our chosen design patterns will be used and document the model, view, control, filter & data access interactions that make up our application along with any data groups moved in each interaction. In doing so, we gain the benefit of viewing our application as sets of coarsely defined, interacting functional processes making it ideal for the application of COSMIC measurement.

3. Application Mapping

3.1. Viewpoint

For the purposes of this study, we size applications only from the *Developer Viewpoint* [5, pp29]. Sizing from the *End-User Viewpoint* gives us an indication of end-user delivered functionality, but does not allow us to take into account the complexities involved in constructing a multi-tiered application which interacts extensively with other systems.

3.2. Identifying Layers

Rollo[l 1] indicates that four layers may be considered when applying COSMIC-FFP to web applications: application, middleware, operating system & device drivers. In his example, the application layer contains the client browser while the middleware layer contains the application server, database and message broker systems. So, if we exclude any extensive use of client-side functionality and the presence of the operating system, the entire web application architecture falls into the middleware layer.

This may seem like a fairly coarse layer structure, but it is dictated by one important principle of layer identification **which** states: *"Software that shares data with other software shall not be considered to be in different layers if they identically interpret the data attributes they share."* [5, pp31]. By this definition, our web application components are considered to be peer components because they interpret the data attributes at the same level of abstraction.

3.3. Identifying Boundaries

The COSMIC-FFP Manual[5, pp33] states that "... *there is a boundary between any two distinct pieces of software in the same layer if they exchange data in peer-to-pcer communications".* It follows, therefore, that HTML Documents, CGI Scripts, JSP's, servlets, Enterprise Java Beans, Java Beans, etc, are all software components **within** this definition.

However, if we treat each such component as a functional process with method calls & returns counted as entry/exit pairs then we are in danger of breaking the COSMIC measurement rules of functional size aggregation[5, pp51] by treating every "building block" of our application as a separate functional process.

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

Rather we must maintain a certain level of abstraction and realise that, for example, while a model process may involve the interaction of several server-side components it remains a single functional process from the measurement point of view. The building blocks that make up that process are an implementation detail which should not concern us at this level. Hence, we must identify the COSMIC boundaries as logical boundaries surrounding our functional processes as opposed to the physical boundaries created by the components implementing those processes.

3.4. Identifying Functional Processes

If we accept the layer & boundary definitions given above, then the functional processes that constitute our web application fall into a single layer and are considered peer processes. However, it is useful to distinguish between the *types* of functional processes we will measure both for the purpose of identification and for later classification **in** our productivity model. Consequently, three types of process - based on the MVC model - can be identified:

- **Control** Processes are processes which are triggered by events from "users" outside the application boundary or other software components within the application boundary, and are responsible for directing application flow. For measurement purposes, we consider a triggering event to initiate a different control [functional] process if the trigger involves the movement of a unique data group across the component boundary, except where a data group is a sub-set of another data group [5, pp36]. Intercepting Filters are considered to be control processes because they too are triggered by an end-user interaction with the application.
- **View** Processes are processes which return data to "users" across the application boundary, carrying out any aggregation and formatting required. A single View Component may encapsulate sev**eral** different view processes, the presentation of which depends on the data group that accompanies it's triggering event. Where this is the case, we would consider each different view to be associated **with** a unique data group and be a distinct functional process. Variations on a single view, such as re-ordering of items in a list based on a data attribute, are considered to be a single functional process.
- **Model Processes** implement business, data access or messaging client logic. These may include simple Java classes, Java Beans or Enterprise Java Beans, JDBC or JMS clients. Each business logic or data access component may encapsulate several distinct functional processes, each defined by the movement of a unique data group across its boundary.

Examples of these different types of functional processes and how they interact within a web application are shown **in** figures 2 and 3.

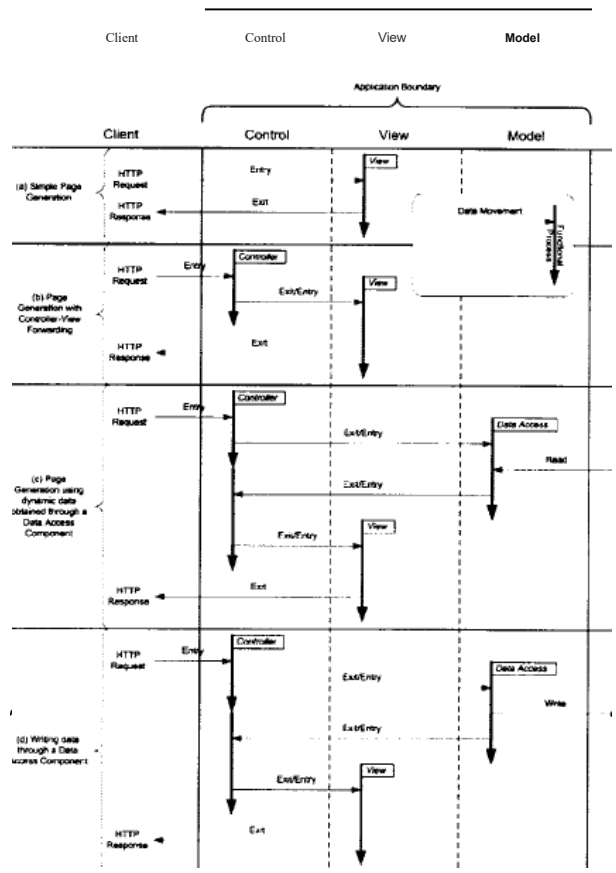

Figure 2. Examples of simple functional process interaction in web applications.

3.5. Identifying Data Attributes and Data Groups

Identifying data groups which pass across the boundary between HTTP Client and Server may be open to some subjective analysis and debate. An initial stance might be that the HTTP Request itself constitutes a single data group, even though it may contain several

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

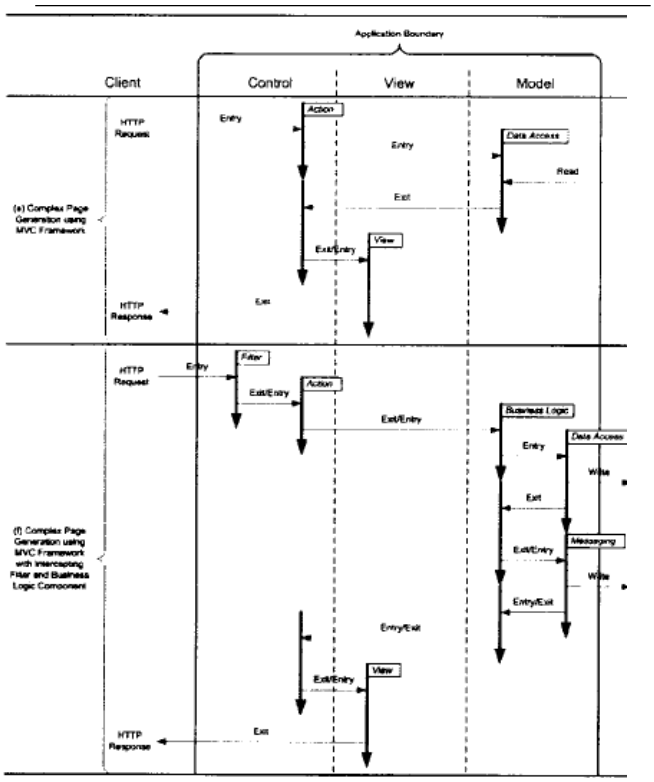

Figure 3. Examples of more complex functional process interaction in web applications.

data attributes such as the target URL, HTTP headers and form or URL-encoded data. For example, a single HTTP Request may contain form data representing a customer's name, address, order details and credit card details. Considering this to be a single data group would oversimplify the data movement and result in a false measurement. Similarly, the HTTP Response could be thought of as being a single data group - the HTML code - even though it may present several different data attributes.

Taking the principle given in [5, pp37] that *"Each data group must be directly related to one object of interest described in the Functional User Requirements ... ",* we can see that the above example should more properly result in a count of 3 data groups (customer details, order details, credit card details). Since they are comprised of data attributes which *"describe or represent"* the user's world, they constitute valid data groups as defined under the definition of data attributes described in [5, pp41].

Whether we classify the HTTP request itself as an additional data group, even if it carries no FUR-referenced data attributes is another matter. Under the definitions presented in [5, pp41] for the identification of data attributes we could classify the attributes of the HTTP Request as either *contextual* or *implementation technical.* If we consider the attributes of the HTTP Request to be implementation technical data then they are invalid for the purposes of identifying data groups, and should not be counted. This would mean, however, that a static web page could not have an entry data movement to trigger it's functional process, since if we consider the HTTP Request alone to have no valid data attributes then there is no data group to constitute a data movement and the triggering event becomes invalid.

We should, therefore, consider the HTTP Request to be contextual data as it can be interpreted under the definition of such to be a *"type of data which specifies what, when or how other data are to be processed by the measured software"* [5, pp41]. This means that even a simple web page would involve the entry data movement of a single data group. The final count for the above example would, therefore, be 4 data groups (3 descriptive and one contextual).

Some consideration should be given to what constitutes a data group in the HTTP Response as well. Here, if we do not consider the HTML code to be a data group in it's own right (as it is simply a presentation mechanism for the client) we have the same problem as we did with the HTTP Request. A static web page responds with no data other than the HTML code itself which, by this definition, is actually an invalid data movement. We should therefore consider the additional data contained within the HTTP Response - HTML code, response headers & cookies - to be contextual data attributes which constitute a data group themselves. This means that even an empty static web page responds with at least one data movement which we consider to be an Exit.

An HTTP Response is also likely to contain several data groups by the definitions given in [5]. Using the above example again, if we sent a response back to our end-user displaying their customer details, order details and payment method, we would again use the principle of associating data groups with "objects of interest from the FUR" and count 4 data groups in this data movement (3 descriptive and 1 contextual).

4. Application Measurement

4.1. Identifying Data Movements

Given the above principles, data movements can now be identified:

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

- Identify an Entry for each data group moved from a user across the boundary into the functional process where it is required [5, pp44]. Thus, we can identify one or more Entry data movements into a control process each time an end-user initiates an HTTP Request and each time a control, view or model process is triggered by another functional process.
- Identify an Exit for each data group moved from a functional process across the boundary to the user that requires it (5, pp44]. Thus we have one or more Exit data movements each time the application sends an HTTP Response back to the end-user and each time a control, view or model process returns control to a triggering process.
- Identify a Read each time a control, view or model process retrieves a data group from persistent storage [5, pp44]. Include in this definition the retrieval of one or more data groups from a messaging system since this is functionally similar to persistent storage from the application's viewpoint.
- Identify a Write data movement each time a control, view or model process saves a data group to persistent storage [5, pp44]. Include in this definition the sending of one or more data groups to a messaging system since this is functionally similar to persistent storage from the application's viewpoint.

4.2. Applying the Measurement Function

With the functional processes and their data movements identified we can now produce a functional size measurement using the principles outlined in [5, pp50j. We count 1 CFSU for each data movement - each instance of an entry, exit, read or write of a data group across a process boundary.

4.3. Adjusted Application Size

A variety of factors may combine to make the components and hence the application more difficult to implement than its raw functional size would suggest. These can be accounted for by adjusting the size through a series of multipliers which reflect the complexity of the application or of individual components or processes. Different factors apply to each component type (view, model, control) and any number of factors can be used as deemed appropriate. Values above 1.0 will increase the effective size of the application and the amount of development effort required while factors less than 1.0 will have the opposite effect. Examples are given in tables 1 lo 3.

For this study, the factors were arrived at through expert judgment in a group discussion, but they could be derived from counts of various artifacts or other metrics [2][3] if required.

Given $SFn > 0$ for each of these factors, the adjusted size can be calculated using equation 1.

$$\text{Size}_0 \quad \text{SizeuIT} \prod_{i=1}^{n} SFi \qquad (1)$$

where Sizea Adjusted size
Sizeu Unadjusted size
SFi Size Factor

MMED	**Multimedia Content:** The number, type and complexity of graphics and other multimedia on the generated page. Accounts for the creation of those graphics and any code required to support them on the page (eg: rollovers). *MMED?:* 1
CLFN	Client-Side Functionality:The amount of client-side functionality present in the view. Includes scripts which carry out input verification, client-side calculations, etc but specifically excludes scripts which affect layout. *CLFN* > 1
LAYC	Layout Complexity: Thecomplexity of the page layout and any interactive elements (eg: scripts) which affect the page layout (through modification of the browser DOM). *LAYC?:* 1
COMP	**Component** Use: The level of black-box or white-box component useto generate the page. This may include the use of JSP Tag Libraries, View Beans, etc. *COMP*> 0

Table 1. Factors affecting the size of View components and functional processes.

5. Productivity

5.1. Unadjusted Productivity

Unadjusted productivity is the baseline productivity of our development team(s) under "normal" circumstances. It represents either a judgment on the part of the team members of the work they can complete in a given unit of time or can be gathered from historic data on completed projects. It is given as the number of CFSU per man-day the development team can implement. Section 9 describes one way to determine base productivity where no historical data is available.

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

STOR	Storage Constraint: A constraint on the component's use of memory or other immediate storage space which require it to be implemented with some efficiency and economy in mind. *STOR* 2: 1
EXEC	**Execution Time Constraint: A** constraint on the time the component should take to execute, usually in order to enable the application to respond to the end-user within a target time. *EX EC* 2: 1
RUSE	**Design for Reuse:** How much the implementation of the component should be considered in order to make the component reusable either by several other components in the application or by other applications. Such considerations may be accounting for interface design, package visibility, inheritance and dependence, etc. *RUSE* 2: 1
RERO	**Reliability & Robustness:** How much effort should be made in the implementation to ensure that the component does not fail (reliability) and that when it does it recovers with little or no impact on the application (robustness). It is reflective of how much attention should be paid in the implementation to parameter checking, use of assertions, error trapping&. handling, etc. *RERO* 2: 1

Table 2. Factors affecting the size of Model components and functional processes

ICOM	**Input Complexity:** The number and type of interactions an end-user can initiate through this component. *!COM* 2: 1
OCOM	**Output Complexity: A** relative increase in the amount and nature of control processing required by the component to determine which view should be used to present data to the end-user. *OCOM* 2: 1

Table 3. Factors affecting the size of Control components and functional processes

5.2. Adjusted Productivity

The circumstances on which our base productivity is based will almost certainly vary from project to project, and so it is necessary to adjust the baseline productivity in line with the particular circumstances of each project. This is done through a series of *Productivity Factors* or multipliers to the base productivity.

The factors presented in tables 4 to 6 are suggestions only. The nature of this model is such that there can be many factors affecting productivity, and their number & nature can be varied as required. All factors should assume a value of 1.0 under normal circumstances and, since the definition of "normal" can vary from one team to another it is left to the individual practitioner to decide what particular set of circumstances constitute baseline normality for their team. Results will be consistent and comparable across projects if that baseline is maintained. Productivity Factor values above 1.0 will increase productivity and decrease the effort required to complete projects while values below 1.0 will have the opposite effect.

Given $PFn > 0$ for each of these factors, the adjusted productivity, $PROD_0$ can be calculated:

$$PRODa \quad PRODuil \prod_{i=1}^{n} PFi \qquad (2)$$

where $PROD_0$ = Adjusted productivity
PRODu Unadjusted productivity
$PF;$ = Productivity Factor

REQV	Requirements Volatility: How rapidly or extensively the project requirements are likely to change forcing re-evaluation of the design of implementation.
SITE	Cross-site Development: Any gain or loss in productivity caused by the development staff being split across multiple geographic sites and having to communicate electronically.
RADP	**Rapid Application Development:** Any change in productivity as a result of the adoption of rapid application development methods or tools.
DOCU	Documentation: How much the availability and accuracy of project documentation may affect team productivity.
PART	Customer Participation: How much direct customer or sponsor participation, in terms of assisting the development team in making design decisions and testing/evaluating results contributes to productivity.
QUAL	**Quality** Constraints: Represents an emphasis on maintaining quality standards and the overheads such activities may create.

Table 4. Factors affecting team productivity.

6. Effort Calculation

Given the size of our application in CFSU and the productivity of our development teams(s) in CFSU per man-day, we can now calculate the effort required to complete the application.

While there are many forms for the relationship between size and cffort[l2] the linear form appears

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

PLAV	Platform Volatility: Any changes in productivity due to changes in the chosen platform (languages, servers, tools, etc) or the fact that decisions about the platform have yet to be made.
DESV	Design **Volatility:** How stable the architectural & technical design is (independent of requirements changes) and how that may affect productivity.
SUPT	Platform Support: How productivity may be helped or hindered by the level of support & guidance provided by the platform vendor. This may mean the availability of consultants on the project team, or simply the level of information carried on the vendor's support weir site.
PLOH	**Platform Overhead:** The overall difficulty of the chosen platform and how architectural, configuration or management requirements may affect productivity.

Table 5. Factors affecting team productivity.

to hold for a large number of projects[13] and provides a simple starting point for initial calculations. COCOMO II, for example, indicates that a mixed square/linear/cube relationship is true for small to medium-sized projects[14]. This model is such that any equation which relates size to productivity can be used. For simplicity in this case, we will assume a basic effort equation of the form:

PCAP	**Programmer Capability:** The capability of the chosen development team(s) relative to the baseline.
PCON	**Personnel Continuity:** How personnel changes or absence throughout the project may affect productivity.
PLAF	**Platform Familiarity: How** familiar the development team(s) are with the chosen platform or architecture and whether there is a learning curve to be climbed which may affect overall productivity.
SDEF	**Software Development Environment** Familiarity: How familiar the team(s) are with their development tools and whether there is another learning curve there.
DOMF	Application Domain Familiarity: How familiar the team(s) are with the application domain and how much they will be able to answer domain questions without referral to the customer.
COMM	Communication: How well the team(s) communicate, not in the sense of the availability of communication methods such as telephone or email but in how easily they are able to communicate concepts and resolve issues.

Table 6. Factors affecting team productivity.

$$\text{Predicted Effort} \quad \frac{SIZE_0}{PROD_0} \qquad (3)$$

where $SIZE_0$ Adjusted size in CFSU

$PROD_0$ Adjusted productivity in CFSU/man-day

7. Effort Probability Function

Using a single value for each of our cost drivers in the effort equation (3) gives us a point-value estimate for the manpower requirement to complete the project and, while this is arguably the "most likely" figure based on the expert judgement or historic data which set the size and productivity values it may not be the statistically most-likely figure. Factors with wide ranges are likely to have a significant effect on our final manpower figure and so we should attempt to calculate the probability of these extreme results for the purposes of determining a more statistically likely result and associated risk identification.

We can obtain a statistically most-likely figure, and generate additional data we can use for risk analysis by using *Monte Carlo Simulation* to generate a probability function for our manpower figure. Monte Carlo Simulation works by randomly choosing sets of cost driver values from the available ranges and calculating application size, productivity and related effort based on those values. If this operation is carried out a significant number of times (say, 1000+) we can use the resulting data to generate a probability distribution for effort indicating how likely a given result is for the input data.

7.1. Monte-Carlo Simulation

In determining our factors, we can use expert judgement, group consensus or analysis of historic data to specify three values for each factor: the lowest expected value, the highest expected value and the most likely expected value. These three values can be used to specify a probability function *(J(x))* for each factor.

In our case study, the triangular distribution was used for simplicity, but ultimately any probability distribution based on the three specified values can be used.

We can then determine a given value of the factor, X, by generating a random number from a uniform distribution U[O, 1] and applying it to the inverse of *f(x)*. By generating several sets of factors randomly and applying them to the effort function3, we can build a series of estimates for our project.

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

Plotting the results on a probability density curve gives us a rough gamma distribution such *as* that in figure 4. This can be used to show the statistically most likely value for effort (the mean of the PDF). Plotting them on a cumulative distribution (figure 5) allows us to carry out some risk analysis, either by starting with a given effort value and determining it's probability, or by choosing a given probability and determining the corresponding effort value.

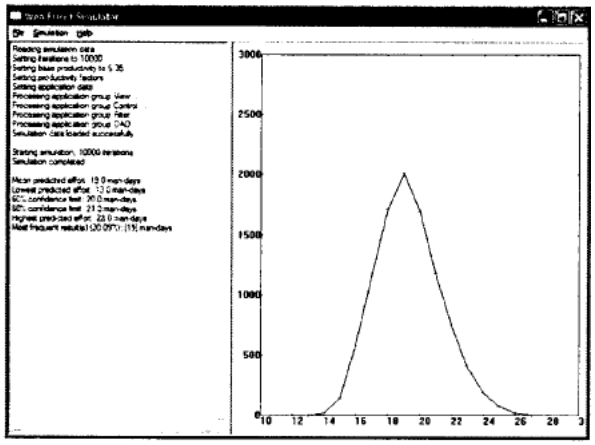

Figure 4. Example PDF generated By Monte Carlo Simulation.

8. Scheduling and Resource Planning

Knowing the effort required to complete a project gives us some idea of the scale of the project but in itself gives us little idea of the calendar time or manpower required.

The COPSEMO model[l4] takes a COCOMO II effort estimate in person-months and predicts the calendar time required to complete the project spread across the four MBASE/RUP phases (Inception, Elaboration, Construction & Transition). While this method remains valid, it assumes that neither schedule or manpower are constrained in any way, predicting a schedule based on principles of best use of manpower across calendar time derived from historic project data.

However, in reality, many projects are constrained either by schedule or available manpower. Where either is specified, we should determine what the other, unconstrained value is and where both are specified we should determine whether the constraints represent a feasible project.

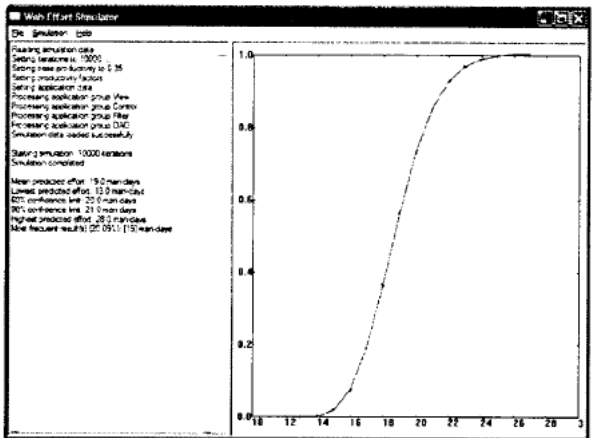

Figure 5. Example CDF generated By Monte Carlo Simulation.

We can do this by starting with either a maximum final schedule (in days) or a maximum number of available personnel and working backwards from these figures using principles defined in the COPSEMO model.

In the *case* of constrained manpower, we assume that the Construction phase uses the specified maximum number of personnel, then derive the manpower requirements and schedule for the other phases relative to that. In the *case* of a constrained schedule we assume that the specified time is 125% of the total schedule and derive the values for the four phases from there. Further calculation then gives us the manpower requirements for the phases.

9. Calibration

9.1. Initial Calibration

Calibrating the model involves establishing a *baseline* relative to the definition of applicable factors. For both application size & productivity, this means establishing the exact circumstances under which each of the applicable factors will be at it's nominal value (1.0). Variations from this baseline encountered in different projects can then be noted *as* higher or lower values for each factor. Note that it is preferable to establish conditions where as many of the factors *as* possible are at their baseline value - the fewer variant fac-

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

tors involved in the final effort calculation, the quicker and easier the calculation will be.

Initial calibration also involves determining a baseline figure for the development team's productivity in CFSU per man-day. If we have no data available from previous projects, we must use a *yardstick:* a set of standard user interactions the functional size of which are known to calibrate our initial productivity estimate.

For each item on our yardstick, expert judgment can be used to determine a man-day estimate and simple calculation then gives us CFSU per man-day for each item and an average across the yardstick as determined by a single developer. Items described on the yardstick can be relatively arbitrary, provided they cover the range of component types that may be developed and their functional size (in CFSU) can be determined accurately.

However, expert judgement, especially that provided by developers, is notorious for being biased towards higher productivity than can actually be achieved. Bias reflected in individual judgements can be mitigated by using various techniques to gather the initial data such as a short-round Delphi assessment, removing outlying results or increasing the sample size to minimise the effect of varying results. The exact techniques used depend on the development team's experience and the size of the samples taken.

The example given in table 7 shows the size in CFSU for each of the items described and the number of man-days each of five developers indicated they would require to complete that item. The mean column gives the mean of all responses for an item, while the adjusted mean is the mean for each item with any responses more than two standard deviations from the unadjusted mean ignored (in this case, none were so the two are identical).

Note that some of the responses are fractional (< 1), indicating that several of these items could be completed in one day. One observation from the results of the questionnaire was that the developer responses tended to be in units of half days or full days rather than specific fractions. This means that some figures appear a little high, others a little low but on the whole the results appear to balance out and "feel" right.

From these responses, we can see that this team expects to complete the 90 CFSU of the example items in 14.17 man-days, giving a base productivity of 6.35 CFSU/man-day.

9.2. Posterior Correction of Model Factors

Once a project is completed, we have data available which can help us refine our initial calibration for future projects. In particular, we are interested in determining which of our sizing and productivity factors were incorrect and, if so, by how much.

Given that we now know how long the project actually took to complete, we can calculate a *correction factor* which is simply the ratio of the actual effort to predicted effort.

$$\text{Correction Factor} \quad f = \frac{\text{Actual Effort}}{\text{Predicted Effort}} \qquad (4)$$

In equation 3 we used a simple linear relationship to calculate effort, and so following on from this we can say that:

$$\text{Corrected Effort} = f \times \sum_{i=1}^{n} \frac{SF_i \times SIZE_{ui}}{F_i \times PR_{ui}} \qquad (5)$$

If we keep our initial estimates of size and productivity constant (assuming that incorrect setting of some of the model factors was responsible for the error in our estimate) we can apply a correction to each of our size and productivity factors such that the ratio of size factors to productivity factors is the same as the ratio of actual effort to predicted effort. We can then determine, for our stated project conditions, what each of the applicable factors *should* have been.

For example, if our predicted effort was 44 man-days but the project took only 41 man-days. Our correction factor is 0.934. We have decided that the layout complexity (LAYC) and execution constraint (EXEC) size factors were not as high as they should have been so we increase both by 10%. We must then adjust our productivity factors so that the correct ratio of size to productivity factor corrections (0.934) is obtained. Here, we have decided that the requirements were more volatile than expected (REQV) and so decreased our productivity, some of the RAD practices we used helped (RADP), some changes in platform (PLAV) that were expected failed to materialise, vendor support was very good (SUPT) and our programmers performed better than expected (PCAP, DOMF).

We can now multiply the minimum, most likely and maximum values of our original size & productivity factors by these corrective values to give a new set of factors. Using these factors, we can recalculate our adjusted size, productivity and hence predicted effort - which now agrees with the actual effort from the completed project.

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

Item	CFSU	Developer Responses A	B	C	D	E	Mean	Adj. Mean
A static web page with pre-defined content using a simpie table layout and minimal graphics. Control over fonts & colours is through a linked CSS style sheet.	2	0.25	0.25	0.33	0.50	0.50	0.37	0.37
A JSP page which derives it's data from the HTTP request, presenting it on-page using a pre-defined tag Iibrary.	2	0.33	0.25	0.50	0.50	0.50	0.42	0.42
A single Struts action class which uses hard-coded data to generate a Java Bean which is forwarded to a JSP page where it is displayed in a simple page layout (as in 2, above) using a pre-defined tag library.	5	1.00	0.75	1.00	1.00	1.00	0.95	0.95
A Data Access Object with four defined read/write functions.	12	1.50	2.00	3.00	1.00	1.00	1.70	1.70
A Struts action class which uses a Data Access Object to retrieve a single data group from the database, then forwards it to a JSP page where it is displayed in a simple page layout (as in 2, above) using a pre-defined tag Iibrary.	10	1.50	2.00	2.00	1.50	1.00	1.60	1.60
A model component which generates three Java Beans from two calls to Data Access Objects.	14	2.00	1.50	3.00	1.00	2.00	1.90	1.90
A Struts action class which uses a model component to generate a Java Bean which then forwards that bean to a JSP page where it is displayed in a simple page layout (as in 2, above) using a pre-defined tag library.	9	1.50	2.00	2.00	1.00	2.00	1.70	1.70
A Struts action class which uses a Data Access Object to retrieve a single data group from the database, then forwards it to one of three JSP pages based on attribute values set in the bean where it is displayed in a simple page layout (as in 2, above) using a pre-defined tag Iibrary.	14	2.00	2.00	3.00	1.50	2.00	2.10	2.10
A model component with three defined functions.	6	1.00	1.20	2.00	0.75	1.50	1.29	1.29
A Struts action class which uses a model component to generate a Java Bean from data retrieved from the database through two Data Access Objects, which then forwards that bean to a JSP page where it is displayed in a simple page layout (as in 2, above) using a pre-defined tag library.	16	2.00	2.00	2.50	1.75	2.50	2.15	2.15
Totals	90							**14.17**

Table 7. Productivity yardstick calibration questions and developer responses.

9.3. Calibration Using Historic Data

While initial, and indeed ongoing, calibration of the model can be carried out by using expert judgement as defined above, it is possible to refine that expert judgement over time by reference to completed projects and historic data.

Once a project is completed we have a record of the [corrected] size & productivity factor values and descriptions of the conditions which determined those values. With this data recorded, it can be used to determine potential factor values for other projects through analogy or statistical aggregation. In other words, if the particular conditions for one or more factors in a new project match those of a completed project then we can use the [corrected] factors from the completed project as a guideline for the values in our new project.

Where we see particular conditions repeated from project-to-project we can combine data available from several projects to produce a mean value for any given factor which we then use in new projects. Alternatively, we can use this mean value to correct our original baseline conditions given that once we see certain conditions repeated more often than not we can assume that they now constitute a new baseline.

10. Case Study

A case study was carried out from September through November 2003 using a web application suite consisting of three related applications: an end-user application, an end-user administrator application and a system administrator application. Developers were asked to note time spent on the project, including their day-to-day activities (such as informal technical discussions related to the project or coffee breaks) but excluding

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

time spent on other projects, supporting live systems or company meetings. A summary of the results is shown in table 8.

Firstly, we can see that the model proved to be quite accurate for the project as a whole. It predicted an effort of 103.84 man-days against an actual effort of 100 man-days, an error of 3.84 man-days and a resulting MRE of only 3.70%. The schedule calculation predicted a development time of 66.88 working days against an actual time of 50 working days, so we over-estimated by 16.88 days (or about 3 weeks). In this case, the schedule over-estimation was considered to be fortuitous as customer-release had been announced for January 2004 (based on this estimate) and the company was able to deliver almost a month early - a fact much appreciated by the customers.

When we look at the results for individual projects, we can see a much greater margin of error. As all three applications had a common database much of it's design was completed as part of the first application, the End-User application. As such, the predicted effort of 22.42 man-days was overrun by a little over a week (30 man-days total), however our schedule required only one developer for Inception & Elaboration while in reality two developers were assigned full-time to the project so the additional effort was recovered and the final schedule was only half a day from our estimate. For this first application, our MRE was therefore 33.81%.

The other two applications resulted in slightly better MREs of 15.25% and 12.2%, both of which represented overestimates. Again, this was due in part to the fact that two developers had been assigned full-time and partly due to the fact that development was found to be easier than expected.

While the applications were sized and estimated separately, it is obvious that there was significant overlap between the applications in terms of development work, particularly during the inception and elaboration phases. Extra time spent here developing a consistent design (particularly for the database) for the three applications resulted in significant savings during construction. When the results for the three applications are combined and the effect these overlapping elements eliminated, the results were extremely encouraging. A prediction error of only 4 man-days across a three month project is believed to be much more accurate than expert judgement had been on previous projects.

11. Conclusions and Future Work

The aim of the project was to produce a repeatable and accurate framework which could be adopted by relatively inexperienced estimators, including the average developer. As such, the emphasis was always on simplicity and practicality. Despite this relatively simple nature, the model has proven (in it's single tested case study so far) to be surprisingly accurate. Given the fact that it had never been applied in a real test case before and was used by a development team who were technically capable but unfamiliar with it, this result is encouraging.

The team readily took to several aspects of the model. The fact that many of the terms used were well known to them made it easy to adopt and discuss application of the model in design meetings. In many cases discussion of the various application size and productivity factors helped the team to anticipate and highlight potential problem areas in the development process.

The general opinion was that the model provided a reasonably accurate estimate in terms of effort but did tend to over-estimate the project schedule - which was not necessarily seen as a bad thing. The use of COSMIC-FFP proved to be the most difficult part of the process, but once the basic concepts and procedure were explained in detail it was seen as being relatively straightforward to apply - although interpretation of the measurement principles was often a point of debate.

Undoubtedly, the model needs to be applied to several more case studies to confirm it's accuracy against a wider range of projects and architectures. Specifically, it's application to different design patterns and architectures other than J2EE (Microsoft .NET, for example) would go some way towards proving it's efficacy in a variety of cases.

Further tool development to remove the reliance on several disparate spreadsheets and simulator code is needed, specifically to bring the process under the control of a single application which can be used to determine, maintain & record project data in a single datastore and further automate the estimation process.

References

[I] Sandeep Chatterjee Ph.D., James Webber Ph.D.,"Developing Enterprise Web Services: An Architect's Guide", *Prentice Hall PTR (November 2003), ISBN 0-13-140160-2*

[2] Fewster, R., Mendes, E., "Measurement, Prediction and Risk Analysis for Web Applications", *Proceedings of*

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.

End-User Application				
Settings		**Item**	**Predicted**	**Actual**
Control Components (CFSU)	49	Development Effort (Man-Days E&C)	19.00	22.00
Filter Components (CFSU)	2	Total Effort (Man-Days 1-T)	22.42	3000
View Components (CFSU)	25	Development Schedule (Days E&C)	11.55	11.00
Model Components (CFSU)	24	Total Schedule (Days 1-T)	14.44	15.00
Total Application Size (CFSU)	100	**Effort Error (man-days)**		**7.58**
Productivity (CFSU/Man-Day)	6.35	**MRE**		**33.81%**
Nominal Effort (Man-Days)	15.75	**Schedule Error (days)**		**-0.56**
Subscriber **Admm Apphcat,on**				
Settings		**Item**	**Predicted**	**Actual**
Control Components (CFSU)	92	Development Effort (Man-Days E&C)	42.00	28.00
Filter Components (CFSU)	2	Total Effort (Man-Days 1-T)	49.56	42.00
View Components (CFSU)	44	Development Schedule (Days E&C)	25.54	14.00
Model Components (CFSU)	76	Total Schedule (Days 1-T)	31.92	21.00
Total Application Size (CFSU)	214	**Effort Error (man-days)**		**-7.56**
Productivity (CFSU/Man-Day)	6.35	**MRE**		**15.25%**
Nominal Effort (Man-Days)	33.70	Schedule Error (days)		**-10.92**
System Admm Apphcat,on				
Settings		**Item**	**Predicted**	**Actual**
Control Components (CFSU)	67	Development Effort (Man-Days E&C)	27.00	20.00
Filter Components (CFSU)	2	Total Effort (Man-Days 1-T)	31.86	28.00
View Components (CFSU)	37	Development Schedule (Days E&C)	16.42	10.00
Model Components (CFSU)	33	Total Schedule (Days I-T)	20.52	14.00
Total Application Size (CFSU)	139	**Effort Error (man-days)**		**-3.86**
Productivity (CFSU/Man-Day)	6.35	**MRE**		**12.12%**
Nominal Effort (Man-Days)	21.89	**Schedule Error (days)**		**-6.52**
ProJect Total				
Settings		**Item**	**Predicted**	**Actual**
Control Components (CFSU)	208	Development Effort (Man-Days E&C)	88.00	70.00
Filter Components (CFSU)	2	Total Effort (Man-Days 1-T)	103.84	100
View Components (CFSU)	106	Development Schedule (Days E&C)	53.51	35.00
Model Components (CFSU)	133	Total Schedule (Days 1-T)	66.88	50.00
Total Application Size (CFSU)	453	**Effort Error (man-days)**		**-3.84**
Productivity (CFSU/Man-Day)	6.35	**MRE**		**3.70%**
Nominal Effort (Man-Days)	71.34	**Schedule Error (days)**		**16.88**

Table 8. Summary of the results of the case study.

IEEE Metrics Symposium -7th International Software metrics Symposium, IEEE CS Press.

[31 Mendes, E., Mosley, N., Counsell, S., "Comparison of Length, complexity and functionality as size measures for predicting Web design and authoring effort", *TEE Proc. Software, (accepted for publication).*

[4] Gary Pollice, Liz Augustine, Chris Lowe, Jas Madhur, "Software Development for Small Teams: A RUP-Centric Approach", *Addison Wesley (January 2004), ISBN 0-321-19950-2*

[5] COSMIC-FFP Measurement Manual version 2.2, *Common Software Measurement International Consortium, January 2003*

[61 Holck, Jesper & Clemmensen, Torkil (2001), "What makes web development different?", *Copenhagen Business School, Dept. of Informmatics, June 2001*

[7] Martin Fowler, David Rice, Matthew Foemmel, Edward Hieatt, Robert Mee, Randy Stafford, "Patterns of Enterprise Application Architecture", *Addison-Wesley {November 2002}, ISBN 0-321-12742-0*

[8] The Apache Struts Web Application Framework http://jakarta.apache.org/struts/

[9] Object Management Group,Unified Modelling Language http://www.uml.org/

[IOI Jim Conallen, "Building Web Applications with UML Second Edition", *Addison-Wesley (October 2002), ISBN 0-201-73038-3*

[11] Tony Rollo, Software Measurement Services UK Ltd, "Sizing E-Commerce", *Presentation to ACOSM 2000*

[12] L. Briand, I. Wieczorek, "Resource Estimation in Software Engineering", *Second edition of the Encyclopedia of Software Engineering, Wiley, NJ, {Editor: J. Marciniak}, ISBN: 0-471-37737-6*

[13] Lionel C. Briand, Khaled El Emam, Frank Bomarius: "COBRA: A Hybrid Method for Software Cost Estimation, Benchmarking, and Risk Assessment", *IEEE International Conference on Software Engineering {/CSE), 1998, Osaka, Japan*

[14] Barry W. Boehm, Chris Abts, A. Winsor Brown, Sunita Chulani, Bradford K. Clark, Ellis Horowitz, Ray Madachy, Donald Reifer, Bert Steece "Software Cost Estimation with COCOMO II", *Prentice Hall (August 2000), ISBN 0-13-026692-2*

Authorized licensed use limited to: Aarhus University. Downloaded on May 23,2025 at 13:04:26 UTC from IEEE Xplore. Restrictions apply.